Security Optimization Assessment — Defender for Endpoint

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — Security Optimization Assessment: Defender for Endpoint 4
- What you get . . . 4
- What this delivers . . . 4
- The standard deployment model . . . 4
- Outcome . . . 5
- Scope boundary . . . 5

1. Solution Overview — Security Optimization Assessment: Defender for Endpoint 5
- What this assessment is . . . 5
- How Defender for Endpoint fits together (what we score) . . . 5
- How the score works . . . 6
- What's in scope . . . 6
- What's out of scope (be honest with buyers) . . . 6
- Authentication and connections used . . . 7
- The standard deployment model . . . 7

2. Requirements — Security Optimization Assessment: Defender for Endpoint 7
- R1. Defender for Endpoint is deployed (even partially) in the tenant . . . 7
- R2. Read access to the Defender for Endpoint API (Security Center) . . . 8
- R3. Advanced hunting enabled and queryable . . . 8
- R4. The in-scope device population is defined . . . 8
- R5. PowerShell modules for the read-only scripts . . . 8
- R6. A named best-practice baseline for scoring . . . 9
- R7. Secure destination for the findings report . . . 9

Global Requirements — Assessments (category-level) 9
- G1. The workload/tenant being assessed . . . 9
- G2. Read-only assessor access (least privilege) . . . 9
- G3. Microsoft Graph / Az / management API access . . . 10
- G4. Licensing for the products in scope . . . 10
- G5. Scope definition and a point-in-time baseline . . . 10
- G6. Best-practice reference baseline . . . 10
- G7. Output handling and confidentiality . . . 10
- G8. Remediation ownership (for the paired remediation) . . . 10

3. Implementation — Security Optimization Assessment: Defender for Endpoint 11
- Stage 0 — Confirm scope and baseline (Assess) . . . 11
- Stage 1 — Establish read-only access (Assess) . . . 11
- Stage 2 — Validate the read (Pilot the read) . . . 12

Stage 3 — Discover the configuration (Assess) . . . 12
Stage 4 — Check coverage per area (Assess) . . . 12
Stage 5 — Score against best practice (Assess) . . . 13
Stage 6 — Generate the findings report (Validate → deliver) . . . 13
Stage 7 — Review and hand off (Expand) . . . 13
5. Testing — Security Optimization Assessment: Defender for Endpoint 13
Gate 1 — Read-only access proven (run `SOAMDE-Validate.ps1`) . . . 13
Gate 2 — Discovery completeness (run `SOAMDE-Discovery.ps1`) . . . 14
Gate 3 — Coverage denominators are real (run `SOAMDE-Coverage.ps1`) . . . 14
Gate 4 — Scoring is defensible (run `SOAMDE-Assess.ps1`) . . . 14
Gate 5 — Report reconciles with the portal (run `SOAMDE-Report.ps1`) . . . 14
Gate 6 — Repeatability (trend) . . . 14
6. Troubleshooting — Security Optimization Assessment: Defender for Endpoint 15
`SOAMDE-Validate.ps1` fails at "connect to Graph" . . . 15
Validate reports "no machines returned" but the portal shows devices . . . 15
Validate flags a write scope present . . . 15
An entire area comes back empty in Discovery . . . 15
Onboarding coverage looks impossibly high (e.g. 100%) . . . 16
Advanced-hunting query returns fewer results than expected / times out . . . 16
Scorecard marks a feature as Fail that the owner says is intentionally not used . . . 16
Report findings don't match the portal on spot-check (Gate 5 fail) . . . 16
Two runs aren't comparable (trend broken) . . . 16
7. Best Practices — Security Optimization Assessment: Defender for Endpoint 17
Keep it strictly read-only . . . 17
Make the numbers defensible . . . 17
Interpret coverage, not just the headline score . . . 17
Hand off well . . . 17
Protect the output . . . 18
Where to go next (adjacent catalog areas) . . . 18
8. Deliverables Summary — Security Optimization Assessment: Defender for Endpoint 18
Documentation . . . 18
Automation artifacts (all read-only) . . . 19
What the buyer receives . . . 19
Outcome . . . 20
Appendix A — PowerShell Scripts 21

Executive Summary — Security Optimization Assessment: Defender for Endpoint

White-label · Assessments

Microsoft Defender for Endpoint is often "deployed" the fast way: licenses are assigned, some devices onboard, EDR shows a few machines, and the project is called done. Months later nobody can answer the questions that actually matter — *how many* in-scope devices are truly onboarded and reporting, is **EDR in block mode** on, are the **attack surface reduction (ASR) rules** in **block** or still in **audit**, is **tamper protection** enforced everywhere, and where is the tenant's **exposure score** bleeding from. A deployed product is not an optimized one, and the gap between the two is exactly where incidents get in.

This solution is a **read-only Security Optimization Assessment**. It discovers your Defender for Endpoint configuration, **scores it against Microsoft best practice**, and hands you a **prioritized, risk-ranked findings report** — coverage (onboarding / EDR block mode / ASR / tamper protection), policy hygiene, and exposure — so you know, with evidence, exactly what to fix and in what order. It **changes nothing**: every script uses read-only Graph Security / Defender / Az scopes, and the only outputs are a scorecard and a findings report.

This is a complete, **run-it-today** package written for an administrator who has used the Defender portal but has never pulled onboarding and ASR posture across a fleet with the API, scored it against a documented baseline, or produced a defensible findings report a security owner can act on.

What you get

- A plain-language **Solution Overview** — what Defender for Endpoint's moving parts are (onboarding, EDR/next-gen protection, ASR rules, tamper protection, web/network protection, and the exposure & Secure Score signals), what "good" looks like for each, and how a read-only assessment turns that into a score.
- A **Requirements Guide** with verify / implement-if-missing for every solution-specific prerequisite (the category-wide ones live in the separate **Global Requirements — Assessments**).
- An **Implementation Guide** of precise, ordered steps — validate read access, discover configuration, run the coverage and scoring passes, generate the findings report, review with the security owner, then re-run to trend.
- Five read-only automation artifacts: a **Discovery** script, an **Assess/score** script, a **per-area Coverage** check, a **Findings Report** generator (HTML + CSV), and an access **Validate** self-check.
- **Testing** and **Troubleshooting** guides so a first run goes cleanly.

What this delivers

A defensible, point-in-time **posture score** for Defender for Endpoint plus a **prioritized remediation backlog**: which devices are unonboarded or unhealthy, which ASR rules are still in audit, where tamper protection is off, which policies conflict or overlap, and where exposure is concentrated — each finding tagged with a severity and a pointer to the fix.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — this product owns the **Assess** stage. The scripts are strictly read-only; **risk-acceptance and remediation priority are the customer's security owners' decisions**, and the fixes themselves are pilot-first work delivered by the paired **Remediation Playbook**. Licensing, plan, and preview facts are flagged **verify** because they change.

Outcome

You stop guessing about your endpoint posture. You get a scored picture and an ordered backlog — the evidence to fix what matters, in the right order, and a repeatable way to re-score and prove the improvement.

Scope boundary

This is an **assessment** — it observes, scores, and reports; it does **not** change your tenant and it does **not** deploy or tune Defender for Endpoint. The fixes are the paired **Remediation Playbook** (pilot-first, `-WhatIf`-gated). Building or onboarding MDE, unified onboarding across Defender workloads, threat hunting, and SOC operations are the existing **DEFXDR**, **DEFCSPM**, **THREATHUNT**, and **SOCXDR** catalog areas — this assessment cross-references them for the remediation path rather than duplicating them.

1. Solution Overview — Security Optimization Assessment: Defender for Endpoint

What this assessment is

This solution is a **read-only Security Optimization Assessment** for **Microsoft Defender for Endpoint (MDE)**. It does not deploy, tune, or change anything. It **discovers** how MDE is configured across your fleet, **scores** that configuration against Microsoft best practice, and produces a **prioritized, risk-ranked findings report**. The deliverable is *evidence and a plan*, not a change — the changes are a separate, pilot-first exercise (the paired **Remediation Playbook**).

The problem it solves is the difference between *deployed* and *optimized*. MDE licensed and partially onboarded still leaves the questions that decide whether it actually works: are all in-scope devices onboarded and healthy; is **EDR in block mode** on; are the **ASR rules** enforcing (block) or merely watching (audit); is **tamper protection** on everywhere; where is exposure concentrated. This assessment answers each with data.

How Defender for Endpoint fits together (what we score)

- **Onboarding & sensor health.** A device is only protected once it is **onboarded** and its sensor is **reporting** ("Active"). Devices that are unonboarded, inactive, or "impaired communication" are blind spots. *We score onboarding coverage and sensor health across the in-scope device population.*
- **Next-generation protection (antivirus) configuration.** Real-time protection, cloud-delivered protection (**MAPS**), **PUA** protection, and signature currency determine whether the AV engine is actually catching things. Misconfigured or passive-mode AV is a common silent gap.

• **EDR in block mode.** When a non-Microsoft AV is primary, **EDR in block mode** lets Defender remediate post-breach malicious artifacts the primary AV missed. It is off by default. *We flag it as a coverage finding when it is not enabled.*

• **Attack surface reduction (ASR) rules.** A set of rules that block common attack techniques (e.g. Office child-process creation, credential theft from LSASS, script/executable behaviors). Their value is only realized in **Block** mode — rules left in **Audit** generate telemetry but stop nothing. *We score how many rules are in Block vs Audit vs Not-configured, and whether they are applied consistently.*

• **Tamper protection.** Prevents attackers (and local admins/malware) from disabling Defender's protections. It is a foundational control; **off** or **inconsistent** tamper protection is a high-severity finding.

• **Web & network protection.** Network protection (block connections to malicious domains/IPs) and web-content filtering. Network protection in audit-only is a partial control.

• **Policy hygiene.** How the above are delivered — Intune endpoint security policies, security baselines, and (legacy) GPO/Configuration Manager. **Conflicting, overlapping, or orphaned** policies produce unpredictable enforcement. *We surface duplication and conflict as hygiene findings.*

• **Exposure & Secure Score.** MDE's **Exposure Score** (from Threat & Vulnerability Management) and the endpoint-related **Microsoft Secure Score** recommendations quantify where risk is concentrated. *We read these signals and fold the top exposure drivers into the ranked findings.*

How the score works

Each assessed area yields **checks** with a defined best-practice expectation (from the baseline named in the report — Microsoft security baselines / Secure Score, *verify against current Microsoft guidance*). Every check is **Pass / Partial / Fail / Not-applicable**, and every non-pass becomes a **finding** with a **severity** (High / Medium / Low), the **evidence** behind it, and a **pointer to the remediation**. The areas roll up into a **posture score** (a percentage of weighted checks passed) so you can trend it over time. The score is a *measurement*, not a verdict — **which findings to accept, defer, or fix, and in what order, is your security owners' decision.**

What's in scope

• Read-only **discovery** of MDE configuration: onboarding & sensor health, AV/next-gen protection settings, EDR block mode, ASR rules, tamper protection, web/network protection, and delivery policies.

• **Scoring** those areas against a named best-practice baseline, producing per-check Pass/Partial/Fail.

• A **per-area coverage** view (which devices/policies are covered vs. gaps).

• Reading **Exposure Score** and endpoint **Secure Score** recommendations to rank findings by risk.

• A **findings report** (HTML + CSV) with severities, evidence, and remediation pointers.

What's out of scope (be honest with buyers)

• **No changes.** This assessment never enables a rule, onboards a device, or edits a policy. Fixing findings is the paired **Remediation Playbook** (pilot-first, `-WhatIf`-gated).

• **Not an MDE deployment / onboarding project.** Standing up MDE and onboarding devices — including unified onboarding across Defender workloads — is the **DEFXDR** area. This assesses what

already exists.

• **Not threat hunting or incident response.** Hunting queries and IR live in the **THREATHUNT** and **SOCXDR** areas. This assessment reads posture, not live threats. *Any live-threat or IR determination is the SOC's decision.*

• **Not a licensing purchase or plan design.** We *read* license/plan state and flag gaps as findings; buying and mapping plans is separate. *Plan/feature mapping changes — verify.*

• **No risk-acceptance decision.** The assessment supplies the score and evidence; accepting or prioritizing risk is the customer's.

Authentication and connections used

• **Microsoft Graph (read-only)** signed in as **Security Reader** / **Global Reader** — for security posture, Secure Score, and (where used) device management configuration. Read scopes such as `SecurityEvents.Read.All`, `SecurityActions.Read.All`, `DeviceManagementConfiguration.Read.All`, `DeviceManagementManagedDevices.Read.All` (per Global Requirements G2/G3). *Verify current scope names.*

• **Microsoft Defender for Endpoint API** (Security Center / `WindowsDefenderATP`) with read roles — machines, recommendations, exposure, and **advanced hunting** (KQL) via `AdvancedHunting.Read.All`. *Verify current API host and permission names.*

• **Az PowerShell / Azure Reader** only where an Azure-scoped signal is read. No write scopes are ever requested.

The standard deployment model

> ***Assess the current MDE configuration (discovery) → configure nothing yet; instead score and rank → pilot the read by running against a representative device scope first → validate the assessor truly has read-only access and the numbers reconcile with the portal → expand to the full fleet, deliver the findings report, and re-run periodically to trend.*** *The repeatable pattern is* ***Assess → Configure → Pilot → Validate → Expand****; this product owns* ***Assess****, and the* ***Configure*** *onward stages are the Remediation Playbook's, done pilot-first.*

2. Requirements — Security Optimization Assessment: Defender for Endpoint

The category-wide prerequisites (the target tenant, least-privilege read-only assessor access, Graph/Az/management API access, licensing awareness, a defined scope and point-in-time baseline, the best-practice reference, secure output handling, and remediation ownership) are in the separate **Global Requirements — Assessments** file — read it first and do not repeat those here. Below are only the prerequisites **specific to assessing Defender for Endpoint**. Each item: **Description / Why required / How to verify / How to implement if missing.**

R1. Defender for Endpoint is deployed (even partially) in the tenant

Description: Microsoft Defender for Endpoint is provisioned and at least some devices are onboarded, so there is a live configuration to read. **Why required:** The assessment scores an existing configuration; a tenant with MDE never provisioned has nothing to enumerate (that is a **deployment** engagement, not an assessment). **How to verify:** Sign in to `security.microsoft.com` → **Assets** → **Devices**; confirm the device inventory is populated. If empty, MDE may not be provisioned. **How to implement if missing:** If MDE is not provisioned at all, use the **DEFXDR** onboarding solution first. Partial coverage is fine and expected — measuring the coverage gap is the point.

R2. Read access to the Defender for Endpoint API (Security Center)

Description: The MDE API (machines, security recommendations, exposure, and advanced hunting) must be reachable with a read-only identity — interactively (delegated) or via an app registration granted read-only application permissions. **Why required:** Onboarding/sensor health, exposure, and KQL hunting posture come from this API, not from Graph alone. **How to verify:** Run `SOAMDE-Validate.ps1` — it confirms it can read machines and run an advanced-hunting query with the current identity. In the portal, confirm you can open **Assets** → **Devices** and **Advanced hunting**. **How to implement if missing:** Assign **Security Reader** (portal RBAC) to the assessor, or create an app registration with **read-only** MDE application permissions (`Machine.Read.All`, `SecurityRecommendation.Read.All`, `Vulnerability.Read.All`, `AdvancedHunting.Read.All`) and admin-consent them. *Verify current permission names and whether your tenant uses Defender unified RBAC.*

R3. Advanced hunting enabled and queryable

Description: MDE **Advanced hunting** (the KQL interface over `DeviceInfo`, `DeviceEvents`, `DeviceTvmSecureConfigurationAssessment`, etc.) is available to the assessor. **Why required:** Several coverage and hygiene checks (onboarding state, ASR events, secure-configuration assessment) are most accurate via KQL. **How to verify:** In `security.microsoft.com` → **Hunting** → **Advanced hunting**, run `DeviceInfo | take 1`. If it returns a row, hunting is available. **How to implement if missing:** Ensure the identity has the read role that includes advanced hunting, and that the tenant retains device data (retention affects how far back queries see). *Verify current retention and licensing for advanced hunting.*

R4. The in-scope device population is defined

Description: A documented definition of *which devices* are in scope (all managed endpoints, a business unit, servers vs. workstations, a platform subset) and the **expected** device count from an authoritative source (Intune/AD/CMDB). **Why required:** Onboarding coverage is only meaningful against an expected denominator — "1,900 onboarded" means nothing without "of how many?". **How to verify:** You have an agreed device count/scope from Intune (`Devices → All devices`), Active Directory, or a CMDB to compare against MDE's inventory. **How to implement if missing:** Export the expected device list before the run; `SOAMDE-Coverage.ps1` accepts an expected-count parameter so onboarding gaps are quantified, not guessed.

R5. PowerShell modules for the read-only scripts

Description: PowerShell 7+ with the **Microsoft.Graph** modules (Authentication, Security, DeviceManagement) available for the scripts; the MDE API is called via Graph/REST with the read scopes. **Why required:** The five automation artifacts are PowerShell and use these modules to collect

configuration read-only. **How to verify:** `Get-Module Microsoft.Graph.Authentication -ListAvailable` returns a version; `Connect-MgGraph` with read scopes succeeds. **How to implement if missing:** `Install-Module Microsoft.Graph -Scope CurrentUser`. *Verify current module versions and required sub-modules.*

R6. A named best-practice baseline for scoring

Description: The specific baseline the score is measured against — Microsoft's **security baselines for Defender for Endpoint**, **Microsoft Secure Score** endpoint recommendations, and/or your organization's approved standard. **Why required:** A score is only meaningful against a stated reference; the report names it so results are defensible and comparable over time. **How to verify:** The baseline is documented in the assessment scope and echoed in the report header. *Baselines evolve — verify against current Microsoft guidance at run time.* **How to implement if missing:** Adopt the Microsoft security baseline + Secure Score endpoint recommendations as the default reference, and note any deliberate organizational deviations as accepted exceptions rather than findings.

R7. Secure destination for the findings report

Description: A restricted location to store the HTML/CSV findings output, which is a map of your endpoint weaknesses. **Why required:** The report is sensitive; it must not be left on a share or emailed broadly. **How to verify:** A need-to-know location exists and access is limited to the security owners. **How to implement if missing:** Provision a restricted location before the run; treat the output as confidential and share on a need-to-know basis. *Risk-acceptance of any finding is the customer's decision.*

Global Requirements — Assessments (category-level)

These requirements apply to every solution in the Assessments category (Security Optimization Assessments, remediation playbooks, and cross-cutting security assessments). Published once at the category level so listings stay small. Confirm these before running any assessment. Each entry: Description / Why / Verify / Implement-if-missing.

> *An assessment observes and scores; it does not change your tenant. Every assessment script here is **read-only** (the paired remediation work is separate and pilot-first). Scope, risk-acceptance, and remediation-priority decisions belong to the customer's security/compliance owners — the assessment supplies the evidence, not the authorization.*

G1. The workload/tenant being assessed

Description: An active Microsoft 365 / Entra tenant and/or Azure subscription(s) hosting the products in scope. Why: There is nothing to assess without the target environment. Verify: Sign in to the relevant portal(s) (`security.microsoft.com`, `portal.azure.com`) as an administrator. Implement: Ensure the target tenant/subscription exists and the products under assessment are deployed (even partially — assessing coverage gaps is the point).

G2. Read-only assessor access (least privilege)

Description: A read-only role sufficient to enumerate configuration — e.g. Security Reader, Global Reader, and/or Reader at the Azure scope. Why: An assessment must never require write access; read-only

proves it changes nothing. Verify: Entra -> `Roles and administrators`; confirm Security Reader / Global Reader (and Azure Reader where cloud resources are assessed). Implement: Assign the read-only roles; do not grant write/admin roles for the assessment itself.

G3. Microsoft Graph / Az / management API access

Description: Microsoft Graph (Security), Azure Resource Manager, and product-specific management APIs reachable with the assessor identity. Why: The assessment scripts collect configuration and posture via these APIs. Verify: `Connect-MgGraph` (read scopes) and `Connect-AzAccount` succeed; the assessment discovery script runs. *Verify current API/permission scopes.* Implement: Consent the read-only Graph scopes; ensure network access to the Microsoft endpoints.

G4. Licensing for the products in scope

Description: The licenses/plans for the products being assessed (e.g. Defender for Endpoint/Identity/Office 365/Cloud Apps plans, Defender for Cloud plans). Why: Some posture/configuration is only present (and only assessable) when the plan is enabled; the assessment flags plan gaps too. Verify: Confirm the relevant plans in the admin/billing portal. *Verify current plan/feature mapping.* Implement: Note unlicensed products as findings (a coverage gap) rather than skipping them silently.

G5. Scope definition and a point-in-time baseline

Description: A defined assessment scope (which products, subscriptions, and users) and the understanding that results are a point-in-time snapshot. Why: Scope keeps the assessment bounded and the score comparable; posture drifts, so results are dated. Verify: The scope is documented; the assessment records the run date. Implement: Agree the scope before running; re-run periodically to trend the score.

G6. Best-practice reference baseline

Description: The best-practice / benchmark the assessment scores against (Microsoft security baselines, Secure Score, CIS/Zero-Trust as applicable). Why: A score is only meaningful against a reference; the assessment states which baseline it uses. Verify: The reference baseline is named in the findings. *Baselines evolve — verify against current Microsoft guidance.* Implement: Use the documented baseline; note where organizational policy intentionally deviates.

G7. Output handling and confidentiality

Description: A place to store the findings report and an understanding that it may contain sensitive posture detail. Why: Assessment output is a roadmap of your weaknesses; it must be handled as sensitive. Verify: A secure destination for the report exists; access is restricted. Implement: Store findings securely; share on a need-to-know basis. *Risk-acceptance of any finding is the customer's decision.*

G8. Remediation ownership (for the paired remediation)

Description: Named owners for acting on findings (the remediation is separate, pilot-first work). Why: An assessment only creates value when someone owns closing the gaps. Verify: Remediation owners are identified; the Remediation Playbook solution is available for the fixes. Implement: Assign owners; use the

Remediation Playbook (pilot-first, -WhatIf-gated) to close findings and re-score.

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation — Security Optimization Assessment: Defender for Endpoint

This is the exact, ordered procedure to run the assessment end to end. It is **read-only** throughout — you create no policies and change no configuration; the only artifacts you create are an assessor identity, a scope file, and the output report. Follow the steps in order and reuse the exact object names below in later steps, testing, and troubleshooting.

Naming convention — every object you create uses `<Purpose> – <Scope> – <Stage>`:

Object	Name to use
Read-only app registration (optional, for unattended runs)	`SOA-MDE-Reader – Tenant – Assess`
Scope definition file	`SOAMDE-Scope – Fleet – Assess.json`
Output run folder	`SOAMDE-Run – <yyyy-mm-dd> – Assess`
Scorecard (CSV)	`SOAMDE-Scorecard – Fleet – Assess.csv`
Findings report (HTML)	`SOAMDE-Findings – Fleet – Assess.html`

Stage 0 — Confirm scope and baseline (Assess)

1. Agree the **in-scope device population** with the owner (all endpoints / a business unit / servers vs. workstations) and record the **expected device count** from Intune, AD, or the CMDB (R4).

2. Record the **baseline** you will score against (Microsoft security baseline + Secure Score endpoint recommendations, and any approved org standard) (R6). *Verify it against current Microsoft guidance at run time.*

3. Create the scope file `SOAMDE-Scope – Fleet – Assess.json` capturing: scope description, expected device count, platform filter (Windows/macOS/Linux/servers), baseline name, and run date. The scripts read this so every run is comparable.

Stage 1 — Establish read-only access (Assess)

1. Sign in to `security.microsoft.com` as the assessor. Under **Settings** → **Microsoft Defender XDR** → **Permissions and roles** (or **Endpoints** → **Roles**, depending on whether unified RBAC is enabled — *verify current layout*), confirm the assessor holds **Security Reader** / a read-only role. Do **not** grant any write/manage role for the assessment.

2. **(Optional, for unattended/scheduled runs)** In Entra → **App registrations** → **New registration**, create `SOA-MDE-Reader – Tenant – Assess`. Under **API permissions**, add **application** read-only permissions: Graph `SecurityEvents.Read.All`, `SecurityActions.Read.All`, `DeviceManagementConfiguration.Read.All`,

`DeviceManagementManagedDevices.Read.All`, and (WindowsDefenderATP) `Machine.Read.All`, `SecurityRecommendation.Read.All`, `Vulnerability.Read.All`, `AdvancedHunting.Read.All`. Click **Grant admin consent**. Add **no** write permissions. *Verify current permission names.*

3. Open PowerShell 7+ and confirm the modules (R5): `Connect-MgGraph -Scopes "SecurityEvents.Read.All","DeviceManagementConfiguration.Read.All","DeviceManagementManagedDevices.Read.All"`. Consent read-only if prompted.

Stage 2 — Validate the read (Pilot the read)

1. Run `SOAMDE-Validate.ps1`. It confirms the current identity can (a) connect to Graph with read scopes, (b) read the MDE machine list, and (c) run a trivial advanced-hunting query — and it asserts that **no write scope** is present. If any check fails, fix access (R2/R3) before continuing; see Troubleshooting.

2. Confirm the numbers reconcile: the machine count `SOAMDE-Validate.ps1` reports should be in the same ballpark as **Assets** → **Devices** in the portal. A large mismatch means a scope or permission problem, not a finding — resolve it first.

Stage 3 — Discover the configuration (Assess)

1. Create the run folder `SOAMDE-Run – <yyyy-mm-dd> – Assess` as the output location (R7, restricted).

2. Run `SOAMDE-Discovery.ps1 -ScopeFile "SOAMDE-Scope – Fleet – Assess.json" -OutputDir "SOAMDE-Run – <yyyy-mm-dd> – Assess"`. It collects, read-only:

• device inventory with **onboarding status** and **sensor health** (Active / Inactive / ImpairedCommunication / NoSensorData);

• **next-gen protection** settings where exposed (real-time / cloud-delivered / PUA), **EDR block mode** state;

• **ASR rules** state per device/policy (Block / Audit / Warn / Not-configured);

• **tamper protection** state;

• **web/network protection** state;

• the delivery **policies** (Intune endpoint security policies / baselines) touching these settings;

• **Exposure Score** and endpoint **Secure Score** recommendations.

Everything is written to the run folder as raw JSON/CSV for the scoring pass. Nothing is changed.

Stage 4 — Check coverage per area (Assess)

1. Run `SOAMDE-Coverage.ps1 -InputDir "SOAMDE-Run – <yyyy-mm-dd> – Assess" -ExpectedDeviceCount <n>`. It computes, per area, the covered-vs-gap picture:

• **Onboarding coverage** = onboarded & Active ÷ expected count (R4), with the list of missing/inactive devices;

- **ASR coverage** = rules in **Block** ÷ recommended rule set, per device and overall, calling out rules stuck in **Audit**;
- **Tamper protection coverage** = devices with tamper protection on ÷ total;
- **EDR block mode / network protection** coverage.

Devices and policies with gaps are listed explicitly so remediation has a work list.

Stage 5 — Score against best practice (Assess)

1. Run `SOAMDE-Assess.ps1 -InputDir "SOAMDE-Run – <yyyy-mm-dd> – Assess" -Baseline "<baseline name>" -OutputCsv "SOAMDE-Scorecard – Fleet – Assess.csv"`. It evaluates each check as **Pass / Partial / Fail / N-A**, assigns a **severity** to each non-pass, rolls the areas up into a **weighted posture score (%)**, and writes the scorecard CSV. The baseline name is stamped into the output so the score is defensible.

Stage 6 — Generate the findings report (Validate → deliver)

1. Run `SOAMDE-Report.ps1 -InputDir "SOAMDE-Run – <yyyy-mm-dd> – Assess" -Scorecard "SOAMDE-Scorecard – Fleet – Assess.csv" -OutputHtml "SOAMDE-Findings – Fleet – Assess.html"`. It produces a **risk-ranked HTML report** (findings sorted High → Low, each with area, evidence, affected count, and a remediation pointer) plus a companion CSV for tracking. The exposure drivers from Stage 3 are folded into the ranking.

2. **Validate the report reconciles** with the portal for a spot-check: pick the top-severity finding and confirm it against the relevant portal blade (e.g. an ASR rule shown as "Audit" in the report should read Audit in the endpoint security policy). This is the assessment's own accuracy gate — see Testing.

Stage 7 — Review and hand off (Expand)

1. Walk the security owner through the report. **Risk-acceptance and remediation priority are their decisions** — annotate accepted exceptions vs. items to fix.

2. Hand the ordered backlog to the paired **Remediation Playbook** (pilot-first, `-WhatIf`-gated) for the fixes. Do **not** make changes from this assessment.

3. **Re-run to trend.** Schedule a periodic re-run (same scope file, new dated run folder) so the posture score is tracked over time and remediation is provably closing gaps.

5. Testing — Security Optimization Assessment: Defender for Endpoint

Because this is a **read-only** assessment, "testing" means two things: proving the assessor truly has **read-only** access (and no write access), and proving the **numbers the report shows are accurate** (they reconcile with the Defender portal). Run these gates in order; each has an explicit **pass/fail**.

Gate 1 — Read-only access proven (run `SOAMDE-Validate.ps1`)

Procedure: Run `SOAMDE-Validate.ps1` with the assessor identity. **Pass:** It reports success on all of — Graph connect with read scopes; MDE machine list readable; a trivial advanced-hunting query (`DeviceInfo | take 1`) returns; and it confirms **no write/manage scope** is present on the token. **Fail:** Any read fails (fix access per R2/R3) **or** a write scope is detected (the assessor is over-privileged — reduce to Security Reader before proceeding; an assessment must not hold write access).

Gate 2 — Discovery completeness (run `SOAMDE-Discovery.ps1`)

Procedure: Run discovery against the scope file; inspect the run folder. **Pass:** Raw outputs exist for every area — devices/onboarding, next-gen protection, EDR block mode, ASR, tamper protection, web/network protection, delivery policies, exposure/Secure Score — and the device count is within a small margin of **Assets** → **Devices** in the portal. **Fail:** An area file is empty or the device count is far from the portal. An empty area usually means a missing read permission or that the feature is unlicensed — record which, and re-run after fixing (an unlicensed feature is itself a finding, not a test failure).

Gate 3 — Coverage denominators are real (run `SOAMDE-Coverage.ps1`)

Procedure: Run coverage with the **expected device count** (R4). **Pass:** Onboarding coverage is expressed as a fraction of the *expected* population (not just of already-onboarded devices), and the gap list names the missing/inactive devices. ASR/tamper coverage percentages are produced with explicit device lists. **Fail:** Coverage is computed against onboarded devices only (denominator wrong) — supply the expected count and re-run, or onboarding gaps will be understated.

Gate 4 — Scoring is defensible (run `SOAMDE-Assess.ps1`)

Procedure: Run the scoring pass; open the scorecard CSV. **Pass:** Every check has a **Pass/Partial/Fail/N-A** result, a **severity** on each non-pass, the **baseline name** stamped in, and the weighted posture score computes. N-A is used (not Fail) for genuinely unlicensed/inapplicable features. **Fail:** Checks are unscored, severities missing, or unlicensed features scored as Fail rather than N-A (flag them as N-A findings instead).

Gate 5 — Report reconciles with the portal (run `SOAMDE-Report.ps1`)

Procedure: Generate the report, then spot-check the **top three** findings against the portal. **Pass:** Each spot-checked finding matches the portal (e.g. an ASR rule the report shows as "Audit" reads Audit in the endpoint security policy; a device the report shows as "Inactive" shows Inactive under Devices). Findings are sorted High → Low with evidence and a remediation pointer. **Fail:** A spot-checked finding does not match the portal — investigate before delivery (usually a stale cache or a scope mismatch; re-run discovery). A findings report must be trustworthy to be actionable.

Gate 6 — Repeatability (trend)

Procedure: Re-run the full sequence against the same scope file into a new dated run folder. **Pass:** The two runs are comparable (same checks, same baseline), so the posture score can be trended and remediation progress is visible. **Fail:** Scope or baseline drifted between runs — pin them in the scope file so trends are valid.

*These gates test the **assessment**, not the tenant's security. A low posture score is a *successful* assessment (it found real gaps); the failure conditions above are about the assessment's own accuracy and read-only guarantee.*

6. Troubleshooting — Security Optimization Assessment: Defender for Endpoint

Common issues when running the read-only assessment, each as **symptom** → **cause** → **fix**. None of these fixes change tenant security configuration — they fix the *assessment run*.

`SOAMDE-Validate.ps1` fails at "connect to Graph"

• **Symptom:** `Connect-MgGraph` errors, or a consent prompt cannot be completed.

• **Cause:** The Microsoft.Graph modules are missing/old, or admin consent for the read scopes has not been granted.

• **Fix:** `Install-Module Microsoft.Graph -Scope CurrentUser` (R5), then reconnect with the read scopes in Implementation Stage 1. For unattended runs, ensure the `SOA-MDE-Reader – Tenant – Assess` app registration has admin-consented read-only permissions.

Validate reports "no machines returned" but the portal shows devices

• **Symptom:** Advanced hunting / machine list is empty from the script, yet **Assets** → **Devices** is populated.

• **Cause:** The identity has Graph read but **not** the MDE (WindowsDefenderATP) read role, or unified Defender RBAC is on and the assessor is not in a read-only role group. *Verify current RBAC model.*

• **Fix:** Assign **Security Reader** in the Defender portal RBAC (or add the app's `Machine.Read.All` / `AdvancedHunting.Read.All` and admin-consent). Re-run Validate.

Validate flags a write scope present

• **Symptom:** Validate fails specifically on the "no write scope" assertion.

• **Cause:** The assessor holds an admin/manage role (e.g. Security Administrator) — over-privileged for an assessment.

• **Fix:** Reduce the identity to **Security Reader** / **Global Reader** before running. An assessment must not hold write access; this is by design, not a bug.

An entire area comes back empty in Discovery

• **Symptom:** One area file (e.g. ASR, tamper protection) is empty while others populate.

• **Cause:** Either the feature/plan is **not licensed/enabled** in the tenant, or a specific read permission for that surface is missing.

• **Fix:** Check licensing/plan (Global Requirement G4) — an unlicensed feature is a **finding** (score it N-A and list it), not a run error. If it is a permission gap, add the specific read scope and re-run. *Verify

current plan/feature mapping.*

Onboarding coverage looks impossibly high (e.g. 100%)

• **Symptom:** Coverage shows near-total onboarding that the owner disputes.

• **Cause:** The denominator is MDE's own inventory, not the **expected** device count (R4) — devices that never onboarded are invisible to MDE, so they cannot lower a self-referential percentage.

• **Fix:** Re-run `SOAMDE-Coverage.ps1 -ExpectedDeviceCount <n>` with the authoritative count from Intune/AD/CMDB. The gap will now include never-onboarded devices.

Advanced-hunting query returns fewer results than expected / times out

• **Symptom:** KQL-based checks under-report or the query errors on volume.

• **Cause:** Data **retention** limits how far back hunting sees, or the query window/scope is too broad for the tenant size.

• **Fix:** Narrow the time window in the scope file, or accept the retention limit and note it in the report. *Verify current advanced-hunting retention for your licensing.*

Scorecard marks a feature as Fail that the owner says is intentionally not used

• **Symptom:** A deliberately-unused control scores as a Fail finding.

• **Cause:** The assessment cannot know an organizational decision to not use a feature.

• **Fix:** Record it as an **accepted exception** in the review (Stage 7). Risk-acceptance is the customer's decision; annotate it so it is not re-flagged as new each run.

Report findings don't match the portal on spot-check (Gate 5 fail)

• **Symptom:** A top finding reads differently in the portal than in the report.

• **Cause:** Stale discovery data (portal changed after the run) or a scope mismatch between the run and the portal filter.

• **Fix:** Re-run `SOAMDE-Discovery.ps1` immediately before reporting for a fresh snapshot, and confirm the portal view uses the same scope. Never deliver a report that fails the reconcile spot-check.

Two runs aren't comparable (trend broken)

• **Symptom:** The posture score jumps in a way that doesn't reflect real change.

• **Cause:** The scope or baseline changed between runs.

• **Fix:** Pin scope and baseline in `SOAMDE-Scope – Fleet – Assess.json` and reuse it; only change them deliberately and note it in the report.

7. Best Practices — Security Optimization Assessment: Defender for Endpoint

Field-proven guidance for running this assessment cleanly, keeping it honest, and turning the output into action.

Keep it strictly read-only

• **Assess with Security Reader, never an admin role.** If the identity can write, the assessment is no longer provably non-disruptive. `SOAMDE-Validate.ps1` deliberately fails if a write scope is present — keep it that way.

• **Never "just fix it" mid-assessment.** The moment you enable a rule you saw was off, the run is no longer a clean point-in-time snapshot and the before/after trend is lost. Record it as a finding; fix it in the Remediation Playbook, pilot-first.

Make the numbers defensible

• **Always supply an expected device count.** Onboarding coverage against "devices MDE already knows about" flatters the result; against the *authoritative* count (Intune/AD/CMDB) it tells the truth.

• **Name the baseline in the report.** "72%" means nothing without "against the Microsoft security baseline + Secure Score endpoint recommendations, on <date>". Baselines evolve — *re-confirm the current guidance each run.*

• **Score unlicensed features as N-A, not Fail** — but list them as coverage findings. An E3 tenant missing an E5-only control is a licensing decision, not a misconfiguration.

• **Reconcile before you deliver.** Spot-check the top findings against the portal every time. One wrong finding undermines the whole report's credibility with the security owner.

Interpret coverage, not just the headline score

• **Audit-mode ASR rules are the classic false-comfort.** A tenant can look "configured" while every rule is in Audit and blocking nothing. Always separate **Block** from **Audit** in the coverage view.

• **Watch sensor health, not just onboarding.** "Onboarded" but Inactive/ImpairedCommunication is a blind spot that headline onboarding numbers hide.

• **Tamper protection off anywhere is high severity** — it undermines every other endpoint control on that device.

Hand off well

• **Deliver an ordered backlog, not a data dump.** The value is the *ranking*: High → Low, with evidence and a remediation pointer, so the owner can work top-down.

• **Let the customer own risk-acceptance.** Some findings will be knowingly accepted (a legacy app that breaks a specific ASR rule). Annotate those as accepted exceptions rather than re-reporting them every run.

• **Re-run on a cadence.** A single score is a snapshot; a trend proves the program is working. Same scope file, new dated run folder.

Protect the output

• The findings report is a **map of your weaknesses** — store it in the restricted location (R7), share on a need-to-know basis, and don't email it broadly.

Where to go next (adjacent catalog areas)

• **Remediation Playbook (SOAREM)** — the paired, pilot-first (`-WhatIf`-gated) solution that *closes* the findings this assessment raises, then you re-score.

• **DEFXDR** — unified onboarding across Defender workloads, if the assessment shows onboarding/coverage gaps at the deployment level.

• **THREATHUNT** — proactive KQL threat hunting once posture is solid.

• **SOCXDR** — SOC onboarding and operations for the alerts MDE generates. *Live-threat and IR determinations belong to the SOC.*

• **Security Optimization Assessment — Defender for Office 365 / Identity / Cloud Apps / Cloud / Defender Suite** — the sibling assessments; run the **Defender Suite** assessment for a single cross-workload posture score.

8. Deliverables Summary — Security Optimization Assessment: Defender for Endpoint

Documentation

Document	Purpose
Executive Summary	The business case — why a deployed MDE is not an optimized one, and what a read-only scored assessment delivers
1. Solution Overview	What MDE's moving parts are (onboarding/sensor health, next-gen protection, EDR block mode, ASR, tamper protection, web/network protection, exposure & Secure Score), what "good" looks like, and how the score works
2. Requirements	Solution-specific prerequisites with Verify / Implement-if-missing (category-wide ones live in Global Requirements — Assessments)
Global Requirements (Assessments)	Category-wide prerequisites, published once
3. Implementation	Exact, ordered, read-only steps — validate access, discover, check coverage, score, generate the findings report, review, and re-run to trend

Document	Purpose
5. Testing	Read-only and accuracy gates with pass/fail — proves the assessment is non-disruptive and its numbers reconcile with the portal
6. Troubleshooting	Common run issues → cause → fix
7. Best Practices	Keep-it-read-only, defensible-numbers guidance and where to go next

Automation artifacts (all read-only)

File	What it does
`SOAMDE-Discovery.ps1`	**PowerShell, read-only.** Collects the full MDE configuration — device inventory with onboarding & sensor health, next-gen protection settings, EDR block mode, ASR rule states, tamper protection, web/network protection, delivery policies, and exposure/Secure Score signals — into a dated run folder as raw JSON/CSV. Changes nothing.
`SOAMDE-Coverage.ps1`	**PowerShell, read-only.** Per-area coverage math — onboarding vs. the *expected* device count, ASR rules in Block vs. Audit, tamper-protection and EDR-block-mode coverage — with explicit gap lists (which devices/policies) for the remediation work list.
`SOAMDE-Assess.ps1`	**PowerShell, read-only.** Scores each check Pass/Partial/Fail/N-A against the named baseline, assigns a severity to every non-pass, and rolls up a weighted posture score (%). Writes the scorecard CSV with the baseline stamped in.
`SOAMDE-Report.ps1`	**PowerShell, read-only.** Generates the risk-ranked findings report (HTML) sorted High → Low with evidence, affected counts, and remediation pointers, plus a companion tracking CSV. Folds in the top exposure drivers.
`SOAMDE-Validate.ps1`	**PowerShell, read-only.** Self-check that the assessor has the read access needed **and no write access** — Graph connect with read scopes, MDE machine list readable, an advanced-hunting query returns, and no write/manage scope present. The assessment's read-only guarantee.

What the buyer receives

- This assessment guide as a **Word** document and **PDF**.
- All five read-only PowerShell scripts in a single ZIP — yours to keep and re-run to trend your posture over time.
- A repeatable process following the model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — this package owns **Assess** (read-only); the fixes are the paired Remediation Playbook, done pilot-first.

Delivered as a Complete Solution Package (PDF + Word + scripts).

Outcome

A defensible, point-in-time **posture score** for Defender for Endpoint and a **prioritized, risk-ranked remediation backlog** — the evidence to fix what matters, in order — plus the scripts to re-score and prove the improvement. Risk-acceptance and remediation priority remain the customer's security owners' decisions.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

SOAMDE-Assess.ps1

Purpose: Evaluates the discovery/coverage evidence as a set of checks, each scored Pass / Partial / Fail / N-A against the named baseline (Microsoft security baseline + Secure Score endpoint recommendations, and/or the org standard). Every non-pass gets a severity (High/Medium/Low) and becomes a finding with evidence and a remediation pointer. The areas roll up into a weighted posture score (%) so it can be trended. Unlicensed/inapplicable features are scored N-A, not Fail. HOW TO RUN pwsh -File .\SOAMDE-Assess.ps1 -InputDir ".\SOAMDE-Run - 2026-07-11 - Assess" -Baseline "Microsoft security baseline + Secure Score (2026-07, verify)" -OutputCsv ".\SOAMDE-Scorecard - Fleet - Assess.csv" Run SOAMDE-Discovery.ps1 and SOAMDE-Coverage.ps1 first. EXPECTED RESULTS A scorecard CSV (one row per check) + findings.json in the run folder, and a posture score to console. .NOTES White-label. Read-only. Scoring only; makes no changes. Risk-acceptance is the customer's decision. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$InputDir, [string]$Baseline = 'Microsoft security baseline + Secure Score endpoint recommendations (verify current guidance)', [Parameter(Mandatory = $true)] [string]$OutputCsv)

How to run: pwsh -File .\SOAMDE-Assess.ps1 -InputDir ".\SOAMDE-Run - 2026-07-11 - Assess" -Baseline "Microsoft security baseline + Secure Score (2026-07, verify)" -OutputCsv ".\SOAMDE-Scorecard - Fleet - Assess.csv" Run SOAMDE-Discovery.ps1 and SOAMDE-Coverage.ps1 first. EXPECTED RESULTS A scorecard CSV (one row per check) + findings.json in the run folder, and a posture score to console. .NOTES White-label. Read-only. Scoring only; makes no changes. Risk-acceptance is the customer's decision. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$InputDir, [string]$Baseline = 'Microsoft security baseline + Secure Score endpoint recommendations (verify current guidance)', [Parameter(Mandatory = $true)] [string]$OutputCsv)

SOAMDE-Coverage.ps1

Purpose: Turns the raw discovery evidence into coverage percentages with explicit gap lists — the work list the remediation will act on. Computes: onboarding coverage against the EXPECTED device count (not just MDE's own inventory), ASR rules in Block vs Audit vs Not-configured, tamper protection coverage, and EDR block-mode / network protection coverage. Names the devices and policies with gaps so nothing is hand-waved. HOW TO RUN pwsh -File .\SOAMDE-Coverage.ps1 -InputDir ".\SOAMDE-Run - 2026-07-11 - Assess" -ExpectedDeviceCount 1900 Run SOAMDE-Discovery.ps1 first. Supply the authoritative expected device count from Intune/AD/CMDB. EXPECTED RESULTS A coverage summary table per area and coverage-*.csv gap lists in the run folder. Read-only. .NOTES White-label. Read-only. Consumes discovery output only; makes no changes. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$InputDir, [int]$ExpectedDeviceCount = 0)

How to run: pwsh -File .\SOAMDE-Coverage.ps1 -InputDir ".\SOAMDE-Run - 2026-07-11 - Assess" -ExpectedDeviceCount 1900 Run SOAMDE-Discovery.ps1 first. Supply the authoritative expected device count from Intune/AD/CMDB. EXPECTED RESULTS A coverage summary table per area and coverage-*.csv gap lists in the run folder. Read-only. .NOTES White-label. Read-only. Consumes discovery output only; makes no changes. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$InputDir, [int]$ExpectedDeviceCount = 0)

SOAMDE-Discovery.ps1

Purpose: Builds the raw evidence base the scoring and coverage passes consume. For the in-scope fleet it collects, read-only: device inventory with onboarding status and sensor health; next-generation protection settings and EDR block-mode state; attack surface reduction (ASR) rule states; tamper protection; web/network protection; the delivery policies (Intune endpoint security policies / baselines) that set these; and the exposure & endpoint Secure Score signals. Nothing is changed. Every area is written as JSON/CSV so the run is reproducible and auditable. HOW TO RUN pwsh -File .\SOAMDE-Discovery.ps1 -ScopeFile ".\SOAMDE-Scope - Fleet - Assess.json" -OutputDir ".\SOAMDE-Run - 2026-07-11 - Assess" Run SOAMDE-Validate.ps1 first. Requires read scopes (Security Reader / Global Reader + MDE read). Uses Microsoft Graph security + device management read APIs and advanced hunting (KQL). Verify current API hosts, permission names, and RBAC model.* EXPECTED RESULTS A populated run folder with one file per area plus a discovery-manifest.json. Read-only — no changes. .NOTES White-label. Read-only. Makes no changes. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$ScopeFile, [Parameter(Mandatory = $true)] [string]$OutputDir, [string[]]$ReadScopes = @('SecurityEvents.Read.All','DeviceManagementConfiguration.Read.All','DeviceManagementManagedDevices.Read.All'))

How to run: pwsh -File .\SOAMDE-Discovery.ps1 -ScopeFile ".\SOAMDE-Scope - Fleet - Assess.json" -OutputDir ".\SOAMDE-Run - 2026-07-11 - Assess" Run SOAMDE-Validate.ps1 first. Requires read scopes (Security Reader / Global Reader + MDE read). Uses Microsoft Graph security + device management read APIs and advanced hunting (KQL). Verify current API hosts, permission names, and RBAC model.* EXPECTED RESULTS A populated run folder with one file per area plus a discovery-manifest.json. Read-only — no changes. .NOTES White-label. Read-only. Makes no changes. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$ScopeFile, [Parameter(Mandatory = $true)] [string]$OutputDir, [string[]]$ReadScopes = @('SecurityEvents.Read.All','DeviceManagementConfiguration.Read.All','DeviceManagementManagedDevices.Read.All'))

SOAMDE-Report.ps1

Purpose: Turns the scorecard and findings into a deliverable: a risk-ranked HTML report (findings sorted High -> Low, each with area, description, result, evidence, affected coverage, and a remediation pointer), the headline posture score and per-area coverage, and a companion tracking CSV. Folds in the top exposure drivers. This is the artifact the security owner reviews and hands to the Remediation Playbook. HOW TO RUN pwsh -File .\SOAMDE-Report.ps1 -InputDir ".\SOAMDE-Run - 2026-07-11 - Assess" -Scorecard ".\SOAMDE-Scorecard - Fleet - Assess.csv" -OutputHtml ".\SOAMDE-Findings - Fleet - Assess.html" Run SOAMDE-Assess.ps1 first. EXPECTED RESULTS An HTML findings report and a findings-tracking.csv. Read-only — reads run output only. .NOTES White-label. Read-only. Store output securely — it maps your weaknesses. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$InputDir, [Parameter(Mandatory = $true)] [string]$Scorecard, [Parameter(Mandatory = $true)] [string]$OutputHtml, [string]$Title = 'Security Optimization Assessment — Defender for Endpoint')

How to run: pwsh -File .\SOAMDE-Report.ps1 -InputDir ".\SOAMDE-Run - 2026-07-11 - Assess" -Scorecard ".\SOAMDE-Scorecard - Fleet - Assess.csv" -OutputHtml ".\SOAMDE-Findings - Fleet - Assess.html" Run SOAMDE-Assess.ps1 first. EXPECTED RESULTS An HTML findings report and a findings-tracking.csv. Read-only — reads run output only. .NOTES White-label. Read-only. Store output securely — it maps your weaknesses. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$InputDir, [Parameter(Mandatory = $true)] [string]$Scorecard, [Parameter(Mandatory = $true)] [string]$OutputHtml, [string]$Title = 'Security Optimization Assessment — Defender for Endpoint')

SOAMDE-Validate.ps1

Purpose: The read-only guarantee of the assessment. Confirms the current identity can (a) connect to Microsoft Graph with read scopes, (b) read the Defender for Endpoint machine list, and (c) run a trivial advanced-hunting query — and asserts that NO write/manage scope is present on the token. If any read fails, fix access before running the rest of the assessment. If a write scope is found, the assessor is over-privileged for an assessment and must be reduced to Security Reader. HOW TO RUN pwsh -File .\SOAMDE-Validate.ps1 Requires the Microsoft.Graph modules. It prompts for interactive read-only consent if needed. For MDE-API checks over REST, sign in so a delegated token for the Defender API is available. Verify current Graph/Defender scope names and RBAC model (unified Defender RBAC vs legacy).* EXPECTED RESULTS A PASS/FAIL line per check and an overall verdict. Read-only — makes no changes. .NOTES White-label. Read-only. Makes no changes. Assessment access self-check. > [CmdletBinding()] param([string[]]$ReadScopes = @('SecurityEvents.Read.All', 'DeviceManagementConfiguration.Read.All', 'DeviceManagementManagedDevices.Read.All'), Write scopes that must NOT be present — an assessment must be read-only. [string[]]$ForbiddenScopePatterns = @('ReadWrite', 'Write', '.Manage', 'FullControl'))

How to run: pwsh -File .\SOAMDE-Validate.ps1 Requires the Microsoft.Graph modules. It prompts for interactive read-only consent if needed. For MDE-API checks over REST, sign in so a delegated token for the Defender API is available. Verify current Graph/Defender scope names and RBAC model (unified Defender RBAC vs legacy).* EXPECTED RESULTS A PASS/FAIL line per check and an overall verdict. Read-only — makes no changes. .NOTES White-label. Read-only. Makes no changes. Assessment access self-check. > [CmdletBinding()] param([string[]]$ReadScopes = @('SecurityEvents.Read.All', 'DeviceManagementConfiguration.Read.All', 'DeviceManagementManagedDevices.Read.All'), Write scopes that must NOT be present — an assessment must be read-only. [string[]]$ForbiddenScopePatterns = @('ReadWrite', 'Write', '.Manage', 'FullControl'))